BUT MMI - CHAMPS-SUR-MARNE

Introduction au JavaScript

COURS	SEMESTRE	ENSEIGNANT	ANNÉE
JavaScript	BUT MMI - S1	Alexis Charpentier	2026 – 2027

Informations pratiques

Pour le bon déroulement du module, voici quelques infos utiles.

Contact : alexis.charpentier@edu.univ-eiffel.fr (ou via Discord)

Ressources : disponibles sur elearning (diapositives, exercices, corrigés)

Évaluation : évaluation finale sur papier, en fin de semestre

Consignes de travail en présentiel :

- Téléphones portables interdits
- Ordinateurs interdits en cours (sauf pour les TD/TP)

UN PEU D'HISTOIRE

Introduction & environnement

HTML/CSS/JS — trio du web

HTML = HyperText Markup Language (1991)

Utilisé pour la structure *(titres, paragraphes, autres éléments)*

CSS = Cascading Style Sheets (1994)

Utilisé pour le style *(couleurs, espacements, mise en page)*

JavaScript = Standard ECMAScript (1995)

Utilisé pour l'interactivité *(ce qui réagit, change, calcule)*

JavaScript s'exécute dans le navigateur

Tout se passe **côté client** (dans Firefox, Chrome, Safari…)

- Chaque navigateur embarque un **moteur JavaScript** qui lit et exécute votre code.
- **Rien à installer** : il suffit d'ouvrir une page web.
- Le moteur lit le code **de haut en bas**, ligne par ligne.

```
console.log('1. je suis lue en premier');
console.log('2. puis moi, ensuite');
console.log('3. et moi en dernier');
```

> Cette année, le JS vit **uniquement dans le navigateur**. C'est déjà tout un monde.

Où écrire du JavaScript ?

Méthode 1 : Deux fichiers côte à côte, reliés par une balise.

La balise `<script src="script.js">` charge votre JavaScript dans la page.

On place le `<script>` **en fin de** `<body>` : le HTML est alors déjà chargé quand le JS s'exécute.

```
<!-- index.html -->
<!doctype html>
<html>
  <body>
    <h1>Mon site</h1>
    <script src="script.js"></script>
  </body>
</html>
```

Méthode 2 : Directement dans la page, entre `<script>` et `</script>`. Cette méthode est réservée aux tests rapides, elle ne permet pas une organisation claire du code.

```
<script>
  console.log('Hello World !');
</script>
```

Votre premier `console.log()`

La console des DevTools, un outil pour tester du JS en direct.

```
// Ouvrez les DevTools (F12 ou Cmd+Option+I), onglet « Console »
console.log('Hello World!');

2 + 2; // la console affiche aussi le résultat : 4
```

- `console.log()` **affiche** ce qu'on lui donne dans la console.
- C'est votre premier outil de **débogage** (comprendre ce que fait le code, identifier les erreurs).

> Réflexe pro : quand « ça ne marche pas », on ouvre la console **avant** tout.

Mettre en place son environnement

Outils de développement

- Navigateur : pour exécuter votre code et voir le résultat.
- IDE (Visual Studio Code) : pour écrire votre code avec de l'aide (coloration, autocomplétion, etc.)
    - Extensions recommandées : Prettier (formatage)

Structure de fichiers

```
.
├── index.html
└── script.js
```

script.js

```
// relié par <script src="script.js">
console.log('script.js est bien chargé ✅ ');
```

LE SOCLE

Variables & types

Déclarer une variable

Une variable, c'est une **étiquette** posée sur une valeur.

```
const reductionRate = 0.2; // une constante : ne changera pas
let price = 10; // une variable : va évoluer

price = 12; // on réaffecte un `let`

console.log(price); // 12
```

`const` par **défaut**, la valeur ne change pas.

`let` quand la valeur **doit changer** au fil du programme.

> On évite `var` : c'est l'ancienne façon de faire, source de bugs de portée.

Conventions de nommage

```
let studentName = 'Ada'; // camelCase : lisible, sans accent ni espace
// let 1name = 'Ada';    // ❌ pas de chiffre en première position
// let prénom = 'Ada';   // ❌ on évite les accents et le français

console.log(studentName); // 'Ada'
```

- Les noms doivent **décrire** la valeur ou l'intention.
- **camelCase** : `finalGrade`, `priceWithTax` — la 1ère lettre minuscule, puis celle de chaque mot en majuscule.
- Pas d'espaces, pas d'accents, pas de caractères spéciaux.
- Certains mots sont réservés par JS : `if`, `for`, `function`, etc. On ne peux pas les utiliser.
- L'anglais est la langue de programmation : `firstName` plutôt que `prenom`.

Connaître le type d'une variable

```
const studentName = 'Ada';
const studentAge = 19;

console.log(typeof studentName); // 'string'
console.log(typeof 'Lovelace'); // 'string'
console.log(typeof studentAge); // 'number'
console.log(typeof 42); // 'number'
console.log(typeof true); // 'boolean'
```

Liste des types JS : `string`, `number`, `boolean`, `undefined`, `object`, `function`.

En fonction du type, certaines opérations sont possibles ou interdites.

> Selon vous, quel opération peut-on faire sur un `number` mais pas sur un `string` ?

Nombres & chaînes de caractères

Les deux types les plus fréquents : `number` et `string`.

`number`

```
const price = 19.99; // number : entier ou décimal
const quantity = 3;
const total = price * quantity; // 59.97
console.log(total); // Affiche 59.97 dans la console

// Opérations : +, -, *, /, %, ** (puissance)
```

`string`

```
const firstName = 'Ada'; // string : entre guillemets ' ' ou " " ou ` `
const lastName = 'Lovelace';
const fullName = firstName + ' ' + lastName; // concaténation avec +
console.log(fullName); // Affiche 'Ada Lovelace' dans la console
```

Les template literals

Assembler du texte sans se noyer dans les `+`.

```
const firstName = 'Ada';
const grade = 15;

// concaténation classique : vite illisible
const a = 'Bravo ' + firstName + ', tu as ' + grade + '/20';

// template literal : entre backticks ` `, variables dans ${ }
const b = `Bravo ${firstName}, tu as ${grade}/20`;

console.log(b); // 'Bravo Ada, tu as 15/20'
```

> Le backtick `` ` `` (AltGr + 7) remplace les `+` : plus lisible, moins d'erreurs.

Booléens : vrai ou faux

Un type à deux valeurs — la base des décisions.

```
const isLoggedIn = true;
const hasPaid = false;

const age = 20;
const isAdult = age >= 18; // une comparaison produit un booléen
console.log(isAdult); // true
```

> Les booléens seront le moteur des **conditions** (section suivante).

À noter, on utilise la convention `is…` ou `has…` pour nommer les variables booléennes (ex. `isLoggedIn`, `hasPaid`, `isEmpty`, `hasPermission`).

États « vides » : `undefined` vs `null`

```
let cart; // déclarée, pas encore de valeur → undefined
console.log(cart); // undefined

let discount = null; // volontairement « vide », à remplir plus tard
console.log(discount); // null

console.log(typeof undefined); // 'undefined'
console.log(typeof null); // 'object' (une bizarrerie historique de JS)
```

`undefined` : JS n'a **pas encore** mis de valeur (« pas défini »).

`null` : **vous** dites volontairement « ici, rien » (« pas de valeur »).

Aperçu : tableaux [ ] & objets {}

Deux façons de regrouper plusieurs valeurs.

```
const colors = ['rouge', 'vert', 'bleu']; // tableau : liste ordonnée
const student = { firstName: 'Ada', age: 19 }; // objet : paires clé/valeur

console.log(colors[0]); // 'rouge'
console.log(student.firstName); // 'Ada'
```

> On voit juste la **syntaxe** ici — on les manipulera en détail en section 5.

Tableaux & objets — la syntaxe

```
// Tableau : des valeurs ordonnées, séparées par des virgules, entre [ ]
const grades = [12, 15, 9, 18];

// Objet : des paires clé: valeur, séparées par des virgules, entre { }
const grade = {
  student: 'Ada Lovelace',
  baseValue: 12,
  maxValue: 20,
  bonus: 3,
};
```

- **Tableau** → quand l'**ordre** compte et que les éléments sont du même genre.
- **Objet** → quand on décrit **une chose** par ses propriétés nommées.

LA LOGIQUE

Structures de contrôle

Comparer des valeurs

Comparer renvoie toujours un **booléen** (`true` / `false`).

```
const grade = 12;

console.log(grade >= 10); // true
console.log(grade < 8); // false
console.log(grade === 12); // true  — strictement égal
console.log(grade !== 20); // true  — différent de
```

- `>`, `<`, `>=`, `<=` pour comparer des nombres.
- `===` pour l'**égalité**, `!==` pour la **différence**.

`==` vs `===`

Deux égalités existent — on n'en utilise **qu'une**.

```
console.log(5 === 5); // true
console.log(5 === '5'); // false — types différents (number vs string)
console.log(5 == '5'); // true  — == convertit avant de comparer 😱
console.log(0 == false); // true  — encore une conversion piégeuse
```

- `==` (faible) : convertit les types avant de comparer → **résultats surprenants**.
- `===` (stricte) : compare valeur **et** type, sans magie.
- Même histoire pour la différence : `!=` convertit, `!==` non.

> Règle simple et sans exception : **toujours** `===` **et** `!==`.

Combiner des conditions

Les opérateurs logiques relient plusieurs comparaisons.

```
const age = 20;
const isRegistered = true;

console.log(age >= 18 && isRegistered); // true
console.log(age < 13 || age > 65); // false
console.log(!isRegistered); // false
```

- `&&` (ET) → vrai si **toutes** les conditions sont vraies.
- `||` (OU) → vrai si **au moins une** l'est.
- `!` (NON) → **inverse** un booléen.

Lire une expression logique

On combine comparaisons et opérateurs pour exprimer une **règle métier**.

```
const age = 16;
const hasPermission = true;

// peut s'inscrire si majeur, OU mineur mais autorisé
const canRegister = age >= 18 || (age >= 16 && hasPermission);

console.log(canRegister); // true
```

- Les **parenthèses** regroupent et clarifient l'ordre de lecture.
- Une condition complexe se lit comme une phrase : « si… et… ou… ».

On remarque la convention de nommage `can…` pour les variables qui expriment une permission ou une capacité (`canRegister`, `canEdit`, `canDelete`).

`if` / `else if` / `else`

Exécuter du code **selon** une condition.

```
const grade = 12;

if (grade >= 10) {
  console.log('Admis');
} else if (grade >= 8) {
  console.log('Rattrapage');
} else {
  console.log('Ajourné');
}
```

La condition entre `( )` doit valoir `true` pour exécuter le bloc `{ }`.

Conditions — points d’attention

```
const hour = 14;

if (hour < 12) {
  console.log('Bonjour ☀️');
} else {
  console.log('Bon après-midi 🌤️');
}
// `else` est optionnel ; `else if` permet d'enchaîner les cas
```

On teste **du plus précis au plus général** (l’ordre des `if` compte).

Pas besoin de `=== true` : `if (isLoggedIn)` suffit si c’est déjà un booléen.

Boucler : `for` & `while`

Répéter des instructions sans copier-coller.

```
// for : quand on connaît le nombre de tours
for (let i = 1; i <= 3; i++) {
  console.log('Tour', i);
}

// while : tant qu'une condition reste vraie
let timeLeft = 3;
while (timeLeft > 0) {
  console.log(timeLeft);
  timeLeft = timeLeft - 1; // ⚠️ faire évoluer la condition !
}
```

`i++` est un raccourci pour `i = i + 1`.

⚠️ **Boucle infinie** : si la condition ne devient jamais fausse, le navigateur se fige.

Parcourir un tableau : `for...of`

La façon la plus lisible de passer sur chaque élément.

```
const firstNames = ['Ada', 'Alan', 'Grace'];

for (const firstName of firstNames) {
  console.log('Bonjour', firstName);
}
// Bonjour Ada
// Bonjour Alan
// Bonjour Grace
```

`for` indexé → `for...of`

Le même résultat, en plus clair.

```
// for...of : on parcourt directement les valeurs, sans index
const firstNames = ['Ada', 'Alan', 'Grace'];
for (const firstName of firstNames) {
  console.log(firstName);
}
```

FACTORISATION DU CODE

Fonctions

Pourquoi des fonctions ?

Pour **nommer** une intention et **éviter de se répéter**.

```
// sans fonction : on recopie la formule à chaque fois
const average1 = (12 + 15 + 9) / 3;
const average2 = (8 + 14 + 11) / 3;

// avec une fonction : on l'écrit une fois, on la réutilise
function average(a, b, c) {
  return (a + b + c) / 3;
}
console.log(average(12, 15, 9)); // 12
```

Bien nommer ses fonctions

Une fonction = **une action**, son nom le dit.

```
function calculateAverage(grades) {
  /* … */
}
function showMessage(text) {
  /* … */
}
function isAdult(age) {
  /* … */
}
```

- Un **verbe** à l'infinitif : `calculate`, `show`, `sort`…
- **Une seule responsabilité** par fonction : plus facile à lire et à corriger.
- Pour les fonctions qui répondent par oui/non : `is…`, `has…` (`isEmpty`, `hasPaid`).

Déclarer & appeler

Deux moments distincts : on **définit**, puis on **utilise**.

```
// 1. on déclare la fonction (on la définit)
function greet() {
  console.log('Bienvenue en MMI !');
}

// 2. on l'appelle (on l'exécute) — autant de fois qu'on veut
greet(); // Bienvenue en MMI !
greet(); // Bienvenue en MMI !
```

> Déclarer ne fait **rien** tant qu'on n'appelle pas avec la fonction.

Déclaration ≠ appel

On ne déclare une fonction **qu'une seule fois**. On peut l'appeler **autant de fois que nécessaire**.

```
function sayHello() {
  console.log('Bonjour 👋');
}

sayHello; // ⚠️ ne fait rien : on n'a pas mis les parenthèses
sayHello(); // ✅ exécute la fonction → Bonjour 👋
```

- Le **corps** `{ }` décrit ce que la fonction fait, mais ne s'exécute pas tout seul.
- Les **parenthèses** `()` déclenchent l'exécution.

On ne déclare une fonction **qu'une seule fois**

On peut l'appeler **autant de fois que nécessaire**.

Paramètres & valeur de retour

Une fonction prend des **entrées** et renvoie un **résultat**.

```
function greet(firstName) {
  return 'Bonjour ' + firstName + ' !';
}

const message = greet('Ada'); // 'Ada' est l'argument
console.log(message); // 'Bonjour Ada !'
```

- `firstName` est un **paramètre** (dans la définition).
- `'Ada'` est l'**argument** (à l'appel).
- `return` renvoie une valeur **réutilisable**.

Dans cet exemple, `greet` est une fonction qui prend un nom en entrée et renvoie un message de bienvenue. On stocke le résultat dans une variable. On affiche ce résultat dans la console.

Plusieurs paramètres & `return`

```
function priceWithTax(netPrice, rate) {
  return netPrice * (1 + rate);
}
console.log(priceWithTax(100, 0.2)); // 120

function show(value) {
  console.log(value); // pas de `return`…
}
const result = show(5); // affiche 5, mais…
console.log(result); // undefined
```

- Plusieurs paramètres : séparés par des **virgules**, dans l'**ordre**.
- Sans `return`, une fonction renvoie `undefined`.

La portée des variables (scope)

Une variable déclarée **dans** une fonction n'existe **que** dedans.

```
function counter() {
  let total = 0; // variable locale
  total = total + 1;
  return total;
}

counter();
console.log(total); // ❌ ReferenceError : total n'existe pas ici
```

Locale vs globale — pourquoi c’est utile

```
let vatRate = 0.2; // globale : accessible partout

function priceWithTax(netPrice) {
  let result = netPrice * (1 + vatRate); // lit la globale, crée une locale
  return result;
}

console.log(priceWithTax(100)); // 120
console.log(result); // ❌ result est local à la fonction
```

- Chaque fonction a sa **bulle** : ses variables ne polluent pas le reste.
- Moins de variables qui traînent = **moins de bugs**, code plus prévisible.

Composer des fonctions

Une fonction peut en **appeler** une autre.

```
function vat(amount) {
  return amount * 0.2;
}

function priceWithTax(netPrice) {
  return netPrice + vat(netPrice); // on réutilise `vat`
}

console.log(priceWithTax(100)); // 120
```

Décomposer un problème

Découper en petites fonctions claires plutôt qu'un gros bloc.

```
function clean(firstName) {
  return firstName.trim(); // enlève les espaces autour
}
function greet(firstName) {
  return `Bonjour ${clean(firstName)} !`;
}

console.log(greet('  Ada  ')); // 'Bonjour Ada !'
```

- Chaque fonction fait **une chose** et porte un **nom parlant**.
- On lit le programme comme une **recette** : étape par étape.
- Plus facile à **tester** et à **réutiliser** ailleurs.

LES DONNÉES

Tableaux & objets

Rappel : `[]` & `{}`

On a vu la syntaxe en section 2 — entrons maintenant dans le **détail**.

```
const colors = ['rouge', 'vert', 'bleu']; // tableau : ordre
const student = { firstName: 'Ada', age: 19 }; // objet : propriétés nommées
```

- **Tableau** `[]` → une **liste** ordonnée de valeurs.
- **Objet** `{}` → un ensemble de **paires clé / valeur**.

Tableaux : les bases

Accéder, mesurer, ajouter, retirer.

```
const cart = ['clavier', 'souris', 'écran'];

console.log(cart[0]); // 'clavier' — l'index commence à 0
console.log(cart.length); // 3 — le nombre d'éléments

cart.push('webcam'); // ajoute à la fin
cart.pop(); // retire le dernier
cart[1] = 'tapis'; // modifie un élément
```

Tableaux — points d'attention

```
const grades = [12, 15, 9];

console.log(grades[0]); // 12 — premier élément (index 0 !)
console.log(grades[grades.length - 1]); // 9  — dernier élément
console.log(grades[10]); // undefined — index inexistant

for (const grade of grades) {
  console.log(grade); // parcours lisible avec for...of
}
```

- L'index commence à **0** → le dernier est à `length - 1`.
- `push` / `pop` **modifient** le tableau existant (même `const`).
- Un index hors limites ne plante pas : il renvoie `undefined`.

Tableaux : les méthodes

Un tableau dispose de **nombreuses méthodes** intégrées — en voici un aperçu.

```
const fruits = ['pomme', 'kiwi', 'fraise'];

fruits.includes('kiwi'); // true — l'élément est-il présent ?
fruits.indexOf('fraise'); // 2 — à quelle position se trouve-t-il ?
fruits.join(' - '); // 'pomme - kiwi - fraise' — assemble en une chaîne de caractères
fruits.slice(1, 3); // ['kiwi', 'fraise'] — extrait une portion
[3, 1, 2].sort(); // [1, 2, 3] — trie les éléments
```

- **Ajouter / retirer :** `push`, `pop`, `unshift`, `shift`
- **Chercher :** `includes`, `indexOf`, `find`
- **Parcourir / transformer :** `forEach`, `map`, `filter`
- **Assembler / réordonner :** `join`, `slice`, `sort`, `reverse`

> Liste **non exhaustive** : le réflexe pro est de chercher « Array methods » sur MDN.

Les fonctions callback

Certaines méthodes attendent une **fonction** en argument : on l'appelle un *callback*.

```
function show(grade) {
  console.log(`Note : ${grade}`);
}

const grades = [12, 8, 15];
grades.forEach(show); // on passe `show`, SANS les parenthèses
```

- On déclare une fonction (`show`) **sans l'appeler** — rappel : `show` ≠ `show()`.
- On la **confie** à `forEach`, qui se charge de **l'appeler pour nous**.
- Elle est exécutée **une fois par élément**, qui lui est passé en argument : `show(12)`, `show(8)`…

> Un *callback* est une fonction qu'on **donne** à une autre, pour qu'elle l'appelle au bon moment.

`forEach` sans magie

Sous le capot, `forEach` n'est qu'une **boucle** qui appelle notre callback.

```
const grades = [12, 8, 15];

// ce que forEach fait pour nous, en coulisses :
for (const grade of grades) {
  show(grade); // il « rappelle » notre callback à chaque tour
}
```

- `forEach` n'est qu'une **boucle** qui appelle notre fonction à chaque élément.
- D'où le nom : la méthode nous « **rappelle** » (*call back*) notre callback.
- `map`, `filter`, `find` font pareil — chacune **exploite** le résultat du callback à sa façon.

Les fonctions anonymes

Un callback peut s'écrire **directement** dans l'appel, sans lui donner de nom.

```
// fonction anonyme : écrite sur place, sans nom
grades.forEach(function (grade) {
  console.log(`Note : ${grade}`);
});
```

- Une fonction **anonyme** n'a pas de nom : on l'écrit **là où on s'en sert**.
- Idéale pour un callback **utilisé une seule fois** — inutile de la déclarer à part.
- Bien **refermer** : `});` — la `}` du callback, puis la `)` de `forEach`.
- Encore plus court avec les **fonctions fléchées** → section 6.

`map` vs `forEach`

Deux méthodes pour passer sur chaque élément — mais dans des buts différents.

```
const grades = [12, 8, 15];

function show(grade) {
  console.log(`Note : ${grade}`);
}
function outOfTwenty(grade) {
  return `${grade}/20`;
}

grades.forEach(show); // exécute show() pour chaque note — ne renvoie rien
const result = grades.map(outOfTwenty); // renvoie un nouveau tableau
console.log(result); // ['12/20', '8/20', '15/20']
```

- `forEach` → pour **agir** à chaque élément (afficher, modifier le DOM). Renvoie `undefined`.
- `map` → pour **transformer** : renvoie un **nouveau tableau** de même longueur.
- La bonne question : je veux **un résultat** (`map`) ou **juste faire une action** (`forEach`) ?

Filtrer & chercher : `filter` & `find`

Ce sont généralement les méthodes les plus utilisées, toutes prennent une **fonction**.

```
const grades = [12, 8, 15, 6, 17];

function isPassing(n) {
  return n >= 10;
}

function isFailing(n) {
  return n < 10;
}

console.log(grades.filter(isPassing)); // [12, 15, 17] — tous ceux qui matchent
console.log(grades.find(isFailing)); // 8 — le premier qui matche
```

- `filter` garde **tous** les éléments dont la fonction renvoie `true` → un **nouveau tableau**.
- `find` renvoie **le premier** élément qui matche (ou `undefined` si aucun).

Objets : accéder & modifier

Décrire une « chose » par ses propriétés.

```
const student = {
  firstName: 'Ada',
  age: 19,
};

console.log(student.firstName); // 'Ada' — accès avec un point
console.log(student['age']); // 19   — ou avec des crochets
student.age = 20; // on modifie une propriété
student.city = 'Lyon'; // on en ajoute une nouvelle
```

Objets — accès & parcours

```
const product = { name: 'Clavier', price: 49, stock: 12 };

console.log(product.price); // 49 — notation `.` (la plus courante)
const key = 'stock';
console.log(product[key]); // 12 — `[ ]` quand la clé est dans une variable

for (const key in product) {
  console.log(key, '=', product[key]); // name = Clavier, price = 49, …
}
```

- `.property` au quotidien ; `["property"]` quand la clé est **dynamique**.
- `for...in` parcourt les **clés** d'un objet (à connaître, usage ponctuel).

Tableau d’objets

Le cas le plus fréquent : une **liste de choses décrites**.

```
const students = [
  { firstName: 'Ada', grade: 15 },
  { firstName: 'Alan', grade: 8 },
  { firstName: 'Grace', grade: 17 },
];

for (const student of students) {
  console.log(`${student.firstName} : ${student.grade}`);
}
```

> Liste d’utilisateurs, de produits, d’articles… c’est partout sur le web.

Filtrer & transformer une liste d’objets

```
const students = [
  { firstName: 'Ada', grade: 15 },
  { firstName: 'Alan', grade: 8 },
  { firstName: 'Grace', grade: 17 },
];

function isPassing(student) {
  return student.grade >= 10;
}
function toFirstName(student) {
  return student.firstName;
}

const passing = students.filter(isPassing); // Ada & Grace
const firstNames = passing.map(toFirstName); // ['Ada', 'Grace']
console.log(firstNames);
```

- On combine `for...of` / `filter` / `map` avec l’**accès aux propriétés**.
- C’est exactement ce qu’on fera pour **afficher des données** dans une page.

L'ABOUTISSEMENT

DOM & événements

Qu’est-ce que le DOM ?

Le navigateur transforme votre HTML en un **arbre d’objets** manipulable.

```
// `document` est la porte d'entrée vers la page
console.log(document.title); // le titre de l'onglet
console.log(document.body); // l'élément <body> et tout son contenu
```

- **DOM** = *Document Object Model*.
- Chaque balise HTML devient un **objet** qu’on peut lire et modifier en JS.

> Modifier le DOM, c’est modifier ce que l’utilisateur **voit** à l’écran.

Le DOM, un arbre d'objets

```
// <body>
//   <h1>Mon site</h1>
//   <ul>
//     <li>Accueil</li>
//     <li>Contact</li>
//   </ul>
// </body>
console.log(document.body.children.length); // 2 : le <h1> et le <ul>
```

- Le HTML est imbriqué → le DOM est un **arbre** : parents, enfants, voisins.
- `document` en est la **racine** côté JavaScript.
- Tout ce qu'on voit dans l'onglet « Elements » des DevTools = le DOM en direct.

Sélectionner des éléments

On vise un élément avec un **sélecteur CSS**.

```
const heading = document.querySelector('h1'); // le 1er <h1>
const button = document.querySelector('#submit'); // par id
const cards = document.querySelectorAll('.card'); // tous les .card

console.log(heading.textContent);
console.log(cards.length); // combien de .card sur la page
```

- `querySelector` → **le premier** élément qui correspond.
- `querySelectorAll` → **tous** les éléments correspondants.

Sélectionner — méthodes & NodeList

```
const cards = document.querySelectorAll('.card'); // → une NodeList
cards.forEach(function (card) {
  console.log(card.textContent); // on parcourt avec forEach
});
```

- `querySelectorAll` renvoie une **NodeList** (≈ un tableau) — pas un seul élément.
- Un **élément** ≠ une **liste d'éléments** : on n'agit pas dessus de la même façon.

On voit ici l'utilité de `forEach` pour parcourir une NodeList et agir sur chaque élément.

Modifier le DOM

Changer le contenu et les classes d'un élément.

```
const heading = document.querySelector('h1');
heading.textContent = 'Bienvenue !'; // change le texte
heading.innerHTML = 'Bienvenue <strong>MMI</strong>'; // injecte du HTML

const card = document.querySelector('.card');
card.classList.add('active'); // ajoute une classe
card.classList.toggle('hidden'); // ajoute / retire selon l'état
```

> Modifier une **classe** plutôt qu'un style : c'est le CSS qui décide du rendu.

Attributs, style & conversions

```
const link = document.querySelector('a');
link.setAttribute('href', 'https://alexischarp.fr'); // écrit un attribut
console.log(link.getAttribute('href')); // le lit
link.style.color = 'crimson'; // style inline (à doser)

const field = document.querySelector('#age'); // un champ <input>
const age = Number(field.value); // ⚠️ .value est TOUJOURS une string, Number() convertit en nombre
console.log(age + 1); // sans Number(), on concatènerait !
```

- `classList` (`add` / `remove` / `toggle`) à privilégier sur `style` inline.
- `textContent` (texte brut, sûr) vs `innerHTML` (interprète le HTML).
- Valeurs de formulaire = **strings** → `Number(...)` ou `parseInt(...)` pour calculer.

Les fonctions fléchées

L'écriture **la plus courte** d'un callback — annoncée en section 5, la voici.

```
const prices = [49, 25, 19];

// corps sur une ligne : ni accolades, ni `return` (retour implicite)
const doubled = prices.map((price) => price * 2);
```

- On remplace le mot `function` par une **flèche** `=>`, placée **après les paramètres**.
- Corps réduit à **une seule expression** → on enlève les `{ }` et le `return` (**retour implicite**).
- Avec **un seul paramètre**, les parenthèses sont facultatives : `price => price * 2`.
- Écriture reine des **callbacks** : `map`, `filter`… et les **événements** (slide suivante).

Écouter un événement

Réagir à une action de l'utilisateur avec `addEventListener`.

```
const button = document.querySelector('#like');

// on « écoute » le clic ; la fonction se déclenche à chaque fois
button.addEventListener('click', (e) => {
  button.textContent = '❤️ Aimé !';
  console.log('Clic sur', e.target);
});
```

`(e) => { ... }` est une **fonction fléchée** (vue juste avant) ; `e` est l'objet `event`, détaillé ensuite.

L'objet `event` & les événements clés

```
const field = document.querySelector('#username');
field.addEventListener('input', (e) => {
  console.log('En direct :', e.target.value); // à chaque frappe
});

const form = document.querySelector('form');
form.addEventListener('submit', (e) => {
  e.preventDefault(); // empêche le rechargement de la page
});
```

Événement	Déclenché quand…
`click`	l'utilisateur clique
`input`	la valeur d'un champ change
`submit`	un formulaire est envoyé

- `e.target` = l'élément concerné ; `e.preventDefault()` annule le comportement par défaut.

INTRODUCTION AU JAVASCRIPT

Fin du module !

De votre premier `console.log()` à la manipulation du DOM, vous avez maintenant les bases pour rendre une page vivante.

- **Le socle** : variables, types, conditions, boucles
- **Structurer** : fonctions, tableaux, objets
- **L'aboutissement** : sélectionner, modifier, réagir aux événements

---

ENSEIGNANT

Alexis Charpentier

CONTACT

alexis.charpentier@edu.univ-eiffel.fr